慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/04/16)

02 - 自然言語処理のための機械学習に入門する

情報工学科 准教授　高道 慎之介

Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	**2026/04/16**	**自然言語処理のための機械学習に入門する**	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習１	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/04	言語モデルを事前学習する	
第10回	2026/06/11	言語モデルを転移学習する	
第11回	2026/06/18	言語を生成 (デコーディング) する	
第12回	2026/06/25	言語やラベルを評価する	
第13回	2026/07/02	自然言語処理から音声言語処理へ	おまけ
第14回	2026/07/09	演習２	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

今日の目的：
線形代数・確率統計・微分積分を復習する

線形代数・確率統計・微分積分を復習する意義

慶應義塾大学 理工学部 情報工学科について短く教えてください.

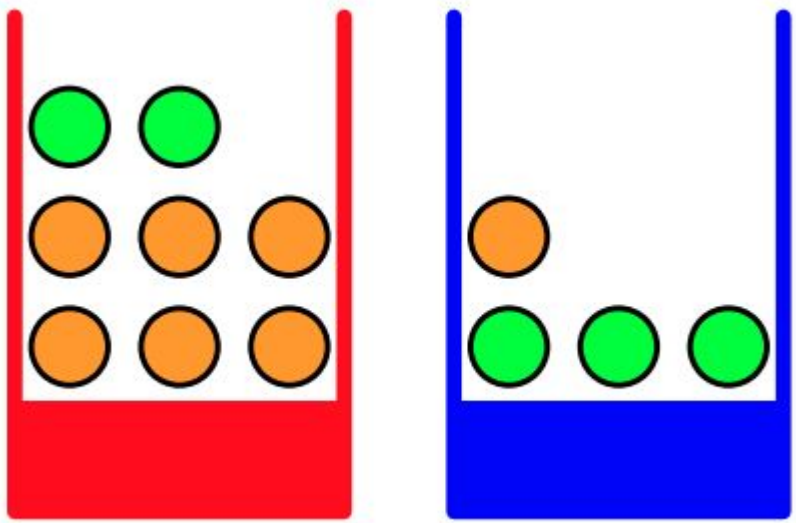
慶應義塾大学 理工学部 情報工学科（Department of Information and Computer Science）は、最先端のコンピュータ技術から、それらを社会に役立てるための知能システムまでを幅広くカバーする ？？？ 学科？コース？

- LLM のような言語モデルは**確率**を表現している.
- 言語モデルはニューラルネットワークで構成されることが多い.
  - ニューラルネットワークは**線形代数**に基づいている.
  - ニューラルネットワークは**微分**を用いて学習される.

復習：確率

確率

- Q. 目を瞑って箱からボールを 1 つ取り出す行為を 10 回繰り返した．とったボールは毎回もとに戻す．赤い箱を 4 回，青い箱を 6 回選んだとき，各箱を選ぶ確率を求めよ．



$$P(x = \text{赤い箱}) = \frac{4}{10}$$

$$P(x = \text{青い箱}) = \frac{6}{10}$$

規格化されているので総和が 1
(連続確率変数の場合は積分が 1)

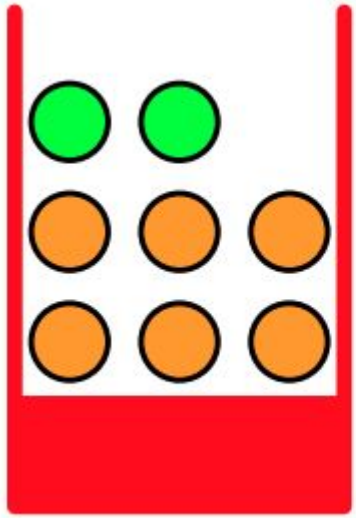
(離散) 確率変数

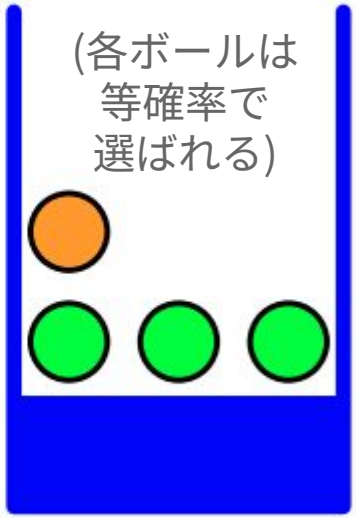
常に非負

https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf

条件付き確率

- Q. 目を瞑って箱からボールを１つ取り出した．選んだ箱が青い箱だったとき，オレンジのボールを選ぶ確率を求めよ．



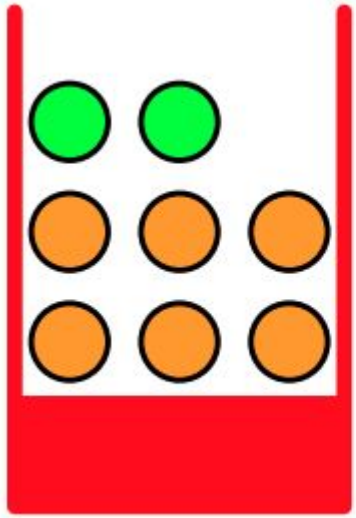


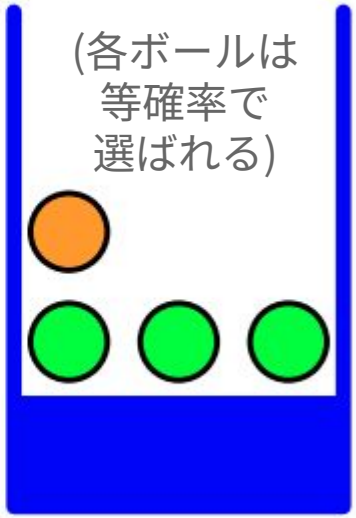
条件付き確率

$$P(y = \text{オレンジ} \mid \underbrace{x = \text{赤い箱}}_{\text{すでに起こった事象（＝条件）}}) = \frac{1}{4}$$

同時確率と連鎖律

- Q. 目を瞑って箱からボールを１つ取り出した．選んだ箱が青い箱であり，かつ，オレンジのボールを選ぶ確率を求めよ．





同時確率

$$P(y = \text{オレンジ}, x = \text{赤い箱})$$

$$= P(y = \text{オレンジ} \mid x = \text{赤い箱})P(x = \text{赤い箱})$$

x かつ y である確率は，x である確率と，
x であった下で y が起こる確率の積．

単語列で考えてみよう

私 / **は** / リンゴ / を / 食べる
彼 / **は** / バナナ / が / 好き / だ
私 / **は** / 公園 / に / 行く
リンゴ / が / 落ちる
私 / **は** / 赤い / リンゴ / を / 買う
空 / **は** / 青い
私 / **は** / 彼 / と / リンゴ / を / 分けた
彼女 / **は** / メロン / を / 食べる
私 / **は** / リンゴ / が / 大好き / だ
犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私}, \text{は}) = \underbrace{P(\text{は}|\text{私})}_{\text{“私” の後に “は” が出現する確率}}\underbrace{P(\text{私})}_{\text{“私” の出現確率}}$$

$$P(\text{私}) = \frac{5}{50}$$ (50 単語のうち 5 回出現)

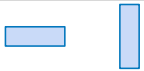
$$P(\text{は}|\text{私}) = \frac{5}{5}$$ (“私” に後続する 5 単語のうち 5 回出現)

連続 3 単語以上にも拡張可能

$$P(\text{私}, \text{は}, \text{彼}) = P(\text{彼}|\text{は}, \text{私})P(\text{は}|\text{私})P(\text{私})$$

復習：線形代数

スカラ・ベクトル・行列，集合

表記	数	図示	表記ルール
a, A	スカラ		イタリック
$\boldsymbol{a}$	ベクトル		イタリック，ボールド，原則小文字
$\boldsymbol{A}$	行列		イタリック，ボールド，大文字

表記	意味	例
$\boldsymbol{A}^\top$	転置	$\boldsymbol{B} = \boldsymbol{A}^\top$ $^\top$ $\boldsymbol{b} = \boldsymbol{a}^\top$ $^\top$

表記	意味	言い換えると
$\mathbb{N}$	自然数	$0, 1, \ldots$
$\mathbb{Z}$	整数	$\ldots, -1, 0, 1, \ldots$
$\mathbb{R}$	実数	$(-\infty, \infty)$
$\mathbb{C}$	複素数	$a + jb$ or $r\exp(j\theta)$ with $a, b, r, \theta \in \mathbb{R}$

ベクトル・行列の和・積

ベクトルの和 (＝要素和)

$$\boldsymbol{a}+\boldsymbol{b}=\begin{bmatrix}a_1\\a_2\\\vdots\\a_n\end{bmatrix}+\begin{bmatrix}b_1\\b_2\\\vdots\\b_n\end{bmatrix}=\begin{bmatrix}a_1+b_1\\a_2+b_2\\\vdots\\a_n+b_n\end{bmatrix}$$

行列の和 (＝要素和)

$$\boldsymbol{A}+\boldsymbol{B}=\begin{bmatrix}a_{11}&\cdots&a_{1n}\\a_{21}&\cdots&a_{2n}\\\vdots&\ddots&\vdots\\a_{m1}&\cdots&a_{mn}\end{bmatrix}+\begin{bmatrix}b_{11}&\cdots&b_{1n}\\b_{21}&\cdots&b_{2n}\\\vdots&\ddots&\vdots\\b_{m1}&\cdots&b_{mn}\end{bmatrix}=\begin{bmatrix}a_{11}+b_{11}&\cdots&a_{1n}+b_{1n}\\a_{21}+b_{21}&\cdots&a_{2n}+b_{2n}\\\vdots&\ddots&\vdots\\a_{m1}+b_{m1}&\cdots&a_{mn}+b_{mn}\end{bmatrix}$$

行列の積

$$\boldsymbol{a}^\top\boldsymbol{b}=a_1b_1+a_2b_2+\cdots+a_nb_n$$

$$=\sum_{i=1}^{n}a_ib_i$$

(アダマール積も覚えておくと便利)

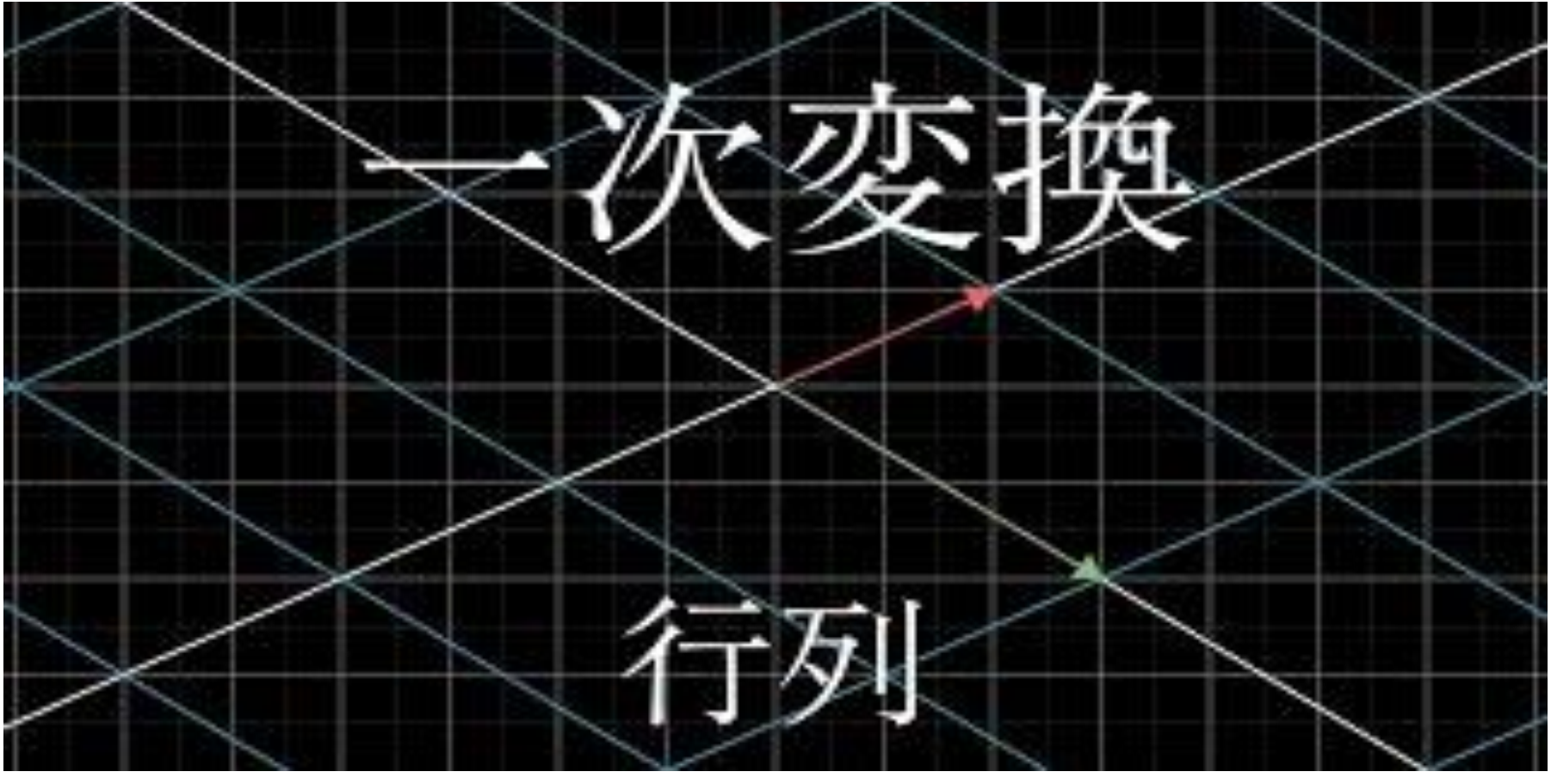
行列の積

$$c_{ij}=\sum_{k=1}^{n}a_{ik}b_{kj}$$

$$\begin{bmatrix}a_{11}&\cdots&a_{1n}\\\vdots&\ddots&\vdots\\a_{i1}&\cdots&a_{in}\\\vdots&\ddots&\vdots\\a_{m1}&\cdots&a_{mn}\end{bmatrix}\begin{bmatrix}b_{11}&\cdots&b_{1j}&\cdots&b_{1p}\\\vdots&\ddots&\vdots&\ddots&\vdots\\b_{n1}&\cdots&b_{nj}&\cdots&b_{np}\end{bmatrix}=\begin{bmatrix}c_{11}&\cdots&c_{1p}\\\vdots&c_{ij}&\vdots\\c_{m1}&\cdots&c_{mp}\end{bmatrix}$$

m n　　n p　　m p

線形変換 $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ とアフィン変換



線形変換 $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ とアフィン変換

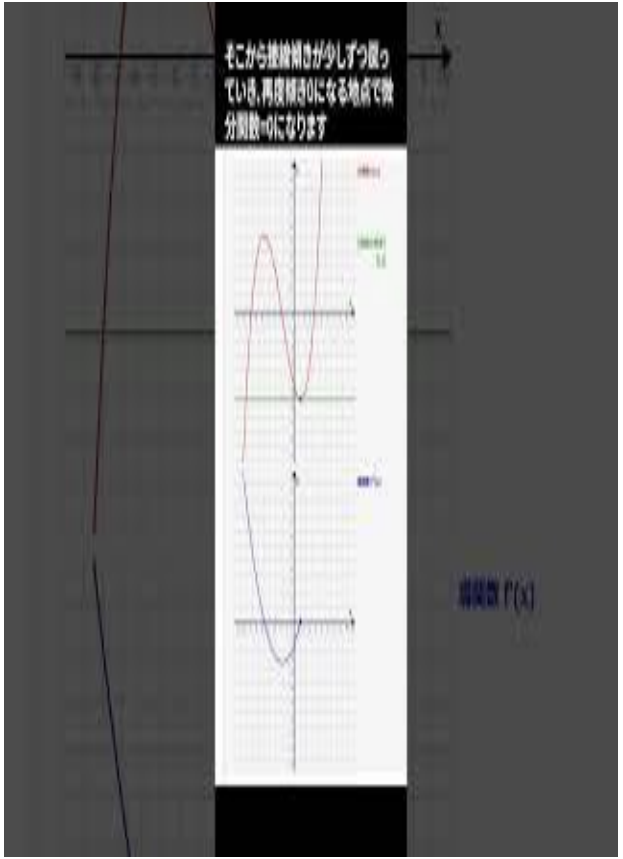
- 線形変換で回転・拡大縮小・せん断を表現できる

- アフィン変換になると，さらに平行移動が可能

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b} \xrightarrow{\text{等価}} \boldsymbol{y} = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{b} \end{bmatrix} \begin{bmatrix} \boldsymbol{x} \\ 1 \end{bmatrix}$$

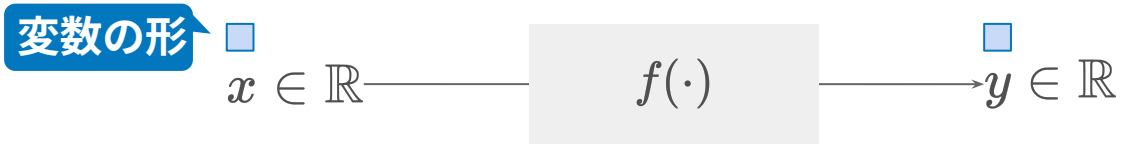
復習：微分

関数の微分



- 微分：変数の微小増分に対する関数の増分
- 変数と関数の種類に基づいて 3 種類を考慮
  - **スカラ**変数をとる**スカラ**値関数
  - **ベクトル**変数をとる**スカラ**値関数
    - ベクトル変数 = 多変数
  - **ベクトル**変数をとる**ベクトル**値関数
- 後述するニューラルネットワークで使用

スカラを引数にとるスカラ値関数

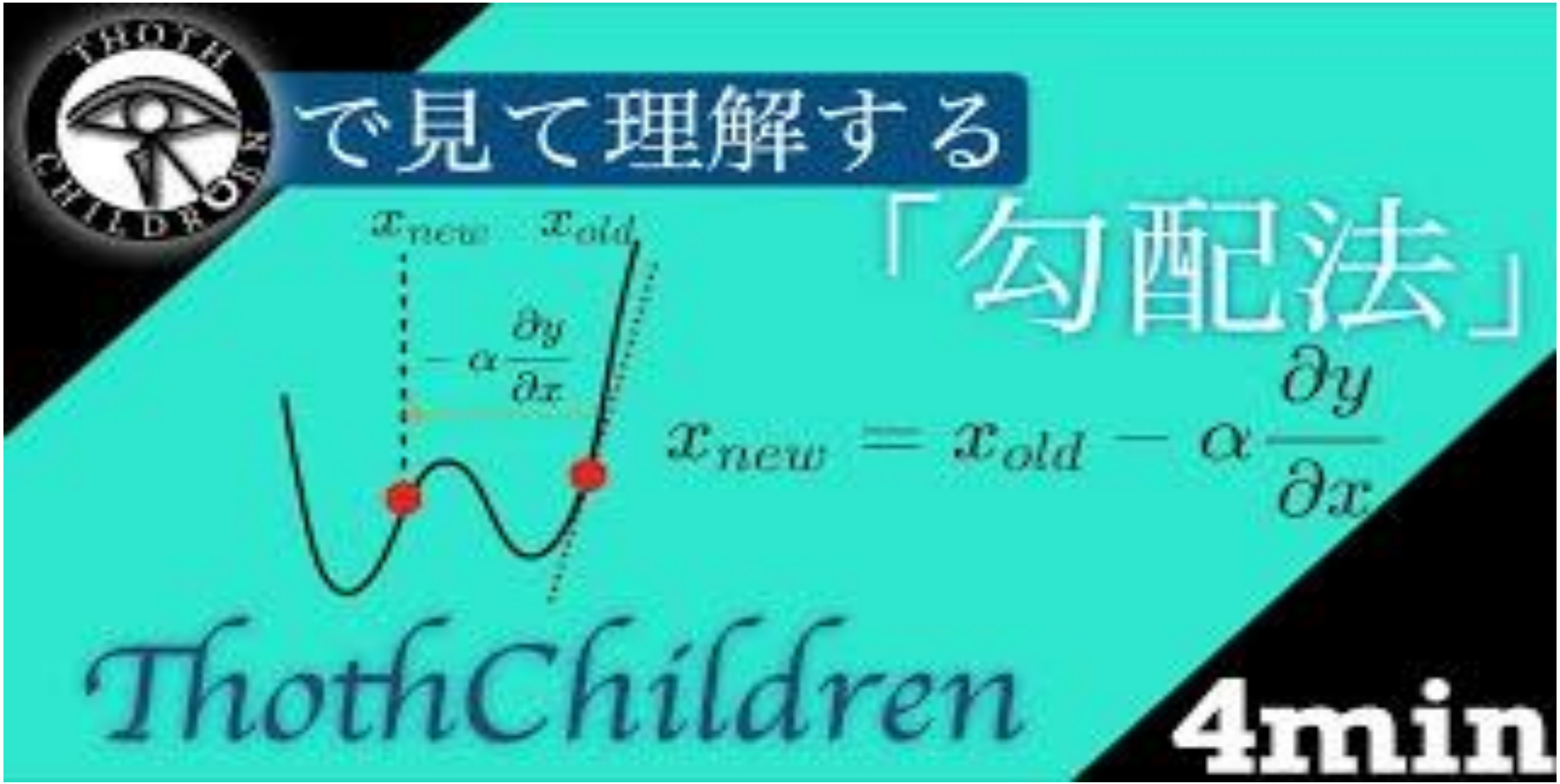


- 関数の例と微分を考える

- 例１：$f(x) = x^2$
  - その微分：$f'(x) = 2x$ 接線の傾き
  - x > 0 で $f'(x) > 0$ (右に向かって増)，x = 0 で $f'(x) = 0$ (最小値)

- 例２：$f(x) = \sin(x)$
  - その微分：$f'(x) = \cos(x)$
  - 変化率が周期的に変化．極小値・極大値が周期的に出現．

例えば勾配法（最急降下法など）に使う

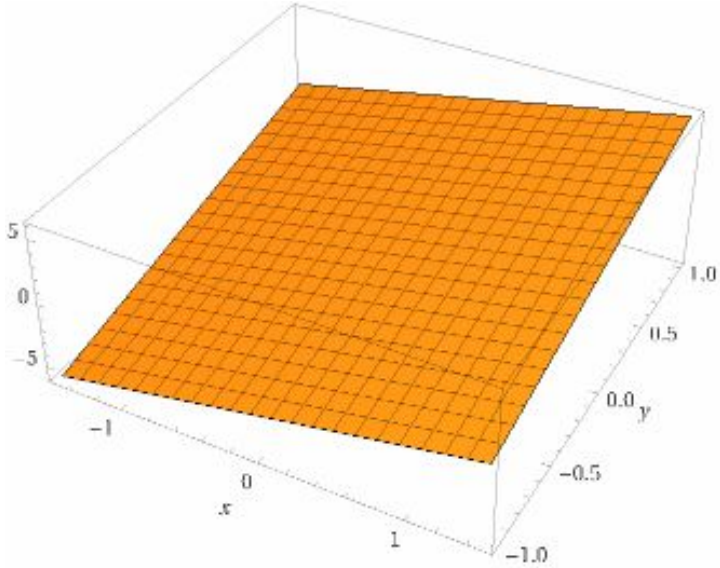


ベクトルを引数にとるスカラ値関数

基本的に列ベクトル

$$\boldsymbol{x} \in \mathbb{R}^N \longrightarrow \boxed{f(\cdot)} \longrightarrow y \in \mathbb{R}$$

- 関数の例を考える
- 例：$f(\boldsymbol{x}) = \boldsymbol{a}^\top \boldsymbol{x}$
  - 書き換えると $f(\boldsymbol{x}) = \sum_i a_i x_i$
  - この(偏)微分は？



https://ja.wolframalpha.com/input?i=%E3%83%97%E3%83%AD%E3%83%83%E3%83%88+2+x+%2B+3+y

スカラ値関数をベクトルで微分する：その定義

- 定義：スカラ値関数をベクトルで微分するとベクトルになる

$$\frac{\partial f}{\partial \boldsymbol{x}} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_d}\right]^\top$$

各次元の偏微分

- 例えば以下のように計算される．公式として覚えておくと楽．

$$f(\boldsymbol{x}) = \boldsymbol{a}^\top \boldsymbol{x}$$

$$\frac{\partial f}{\partial x_i} = a_i \longrightarrow \frac{\partial f}{\partial \boldsymbol{x}} = \boldsymbol{a}$$

$$f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{x}$$

$$\frac{\partial f}{\partial x_i} = \frac{\partial}{\partial x_i} \sum_{j=1}^{d} x_j^2 = 2x_i \longrightarrow \frac{\partial f}{\partial \boldsymbol{x}} = 2\boldsymbol{x}$$

ベクトルを引数にとるベクトル値関数

$$\boldsymbol{x} \in \mathbb{R}^N \longrightarrow \boxed{f(\cdot)} \longrightarrow \boldsymbol{y} \in \mathbb{R}^M$$

- 例えば

$$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$$

深層学習における線形層

- その微分は行列 (ヤコビ行列，ヤコビアン) になる

$$\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \left[\frac{\partial f_i}{\partial x_j}\right]_{ij}$$

出力側が列，入力側が行になるので注意．
(スカラ値ベクトルのときは，入力側が列)

例えば

- 前ページの例なら

$$\boldsymbol{x} \in \mathbb{R}^N \longrightarrow \boxed{\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}} \longrightarrow \boldsymbol{y} \in \mathbb{R}^M$$

$$\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \boldsymbol{A}$$

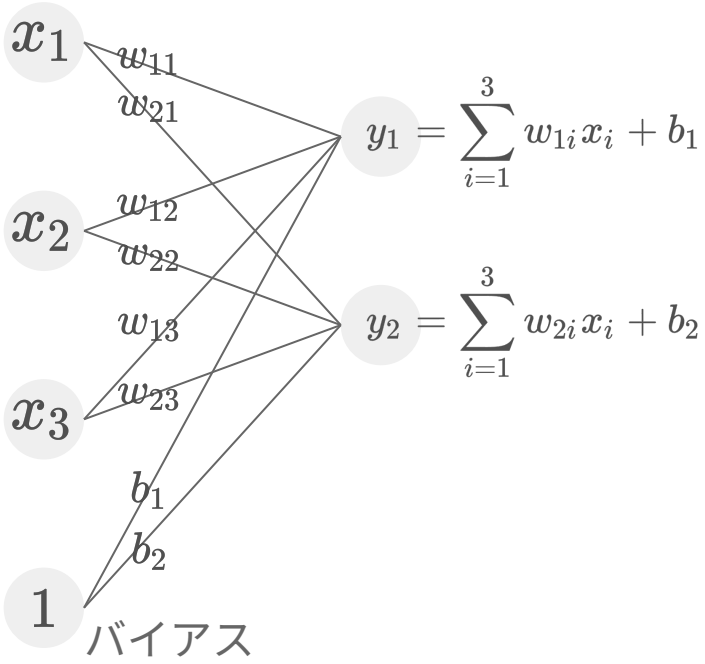
- 連鎖律 (chain rule) も同様に適用可能
  - 信号処理システムを連結する．ニューラルネットワークを連結する

$$\boldsymbol{x} \in \mathbb{R}^N \longrightarrow \boxed{\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}} \rightarrow \boldsymbol{y} \in \mathbb{R}^M \rightarrow \boxed{\boldsymbol{g}(\boldsymbol{y}) = \boldsymbol{C}\boldsymbol{y} + \boldsymbol{d}} \rightarrow \boldsymbol{z} \in \mathbb{R}^L$$

$$\frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{f}} \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \boldsymbol{C}\boldsymbol{A}$$

ニューラルネットワーク

単純パーセプトロン



- 入力の各値を重みづけて足し合わせ
- つまるところアフィン変換

$$\begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$$

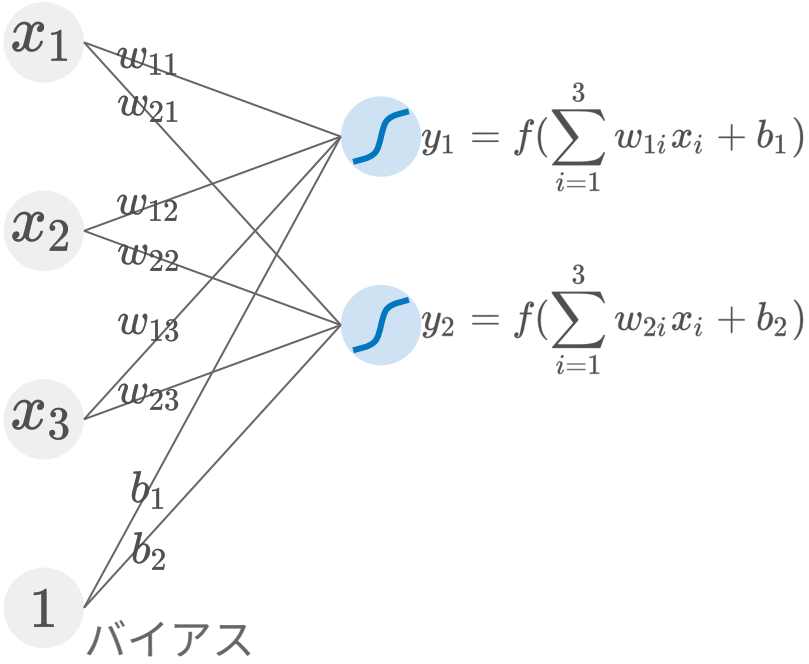
$$\boldsymbol{y} = [\boldsymbol{W}, \boldsymbol{b}][\boldsymbol{x}^\top, 1]^\top$$

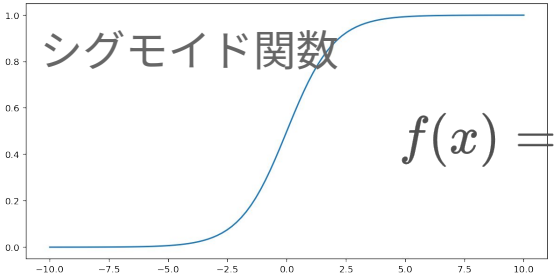
- これだけだと多層でもアフィン変換

$$\boldsymbol{y} = \boldsymbol{W}_N \cdots \boldsymbol{W}_2 \boldsymbol{W}_1 \boldsymbol{x} = \boldsymbol{A}\boldsymbol{x}$$

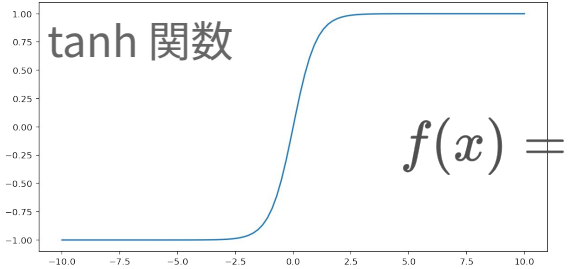
- **活性化関数で非線形化**

活性化関数による非線形変換

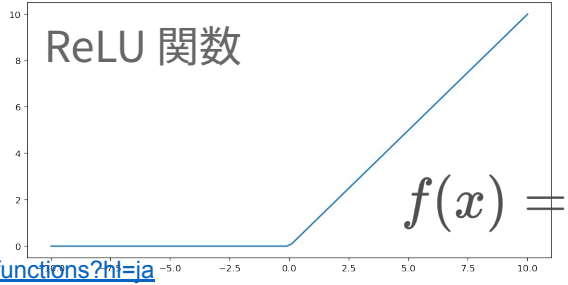




$$f(x) = \frac{1}{1+e^{-x}}$$



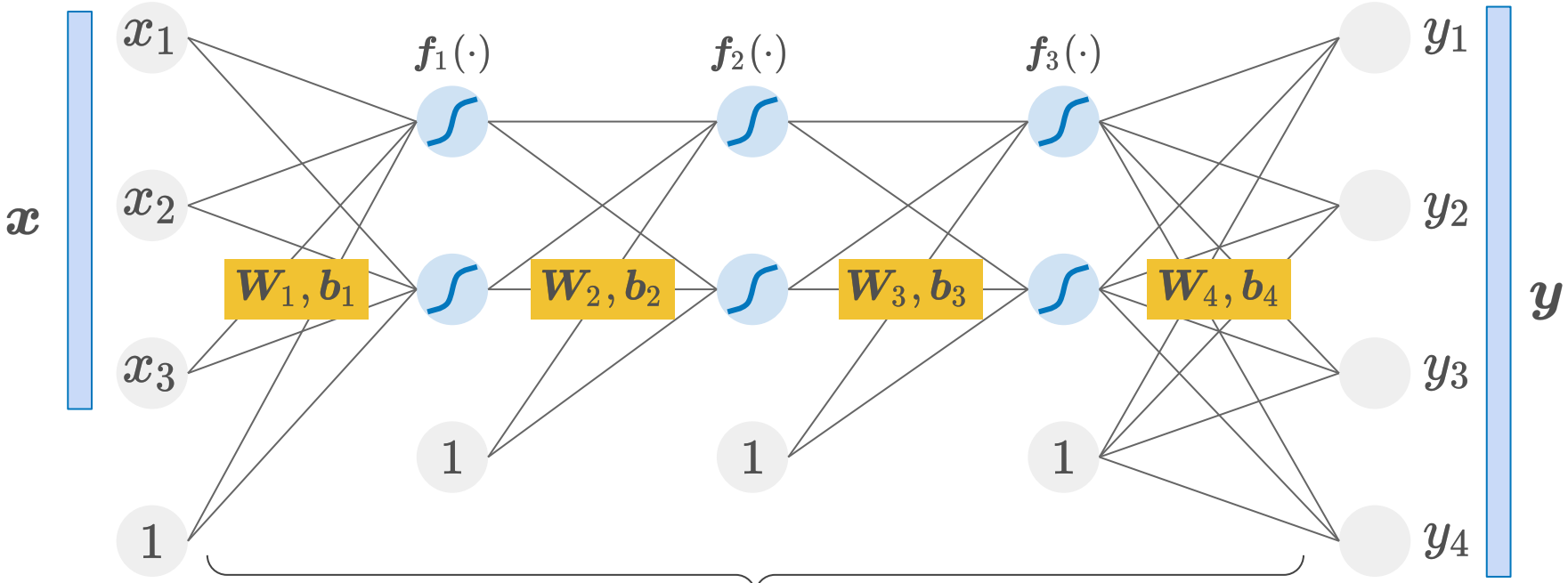
$$f(x) = \tanh(x)$$



$$f(x) = \max(0, x)$$

https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions?hl=ja

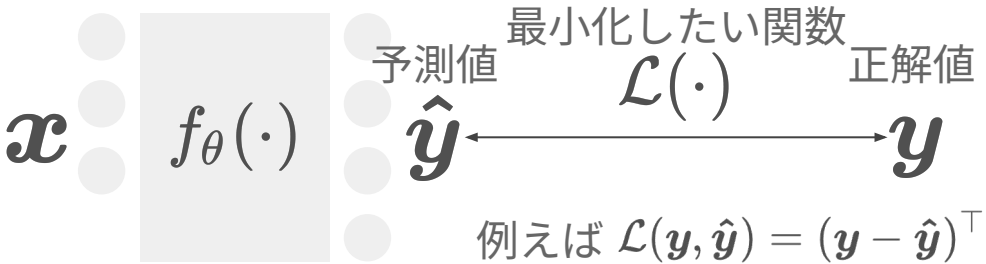
多層パーセプトロン



$\boldsymbol{y} = f_{\theta}(\boldsymbol{x})$ と表記

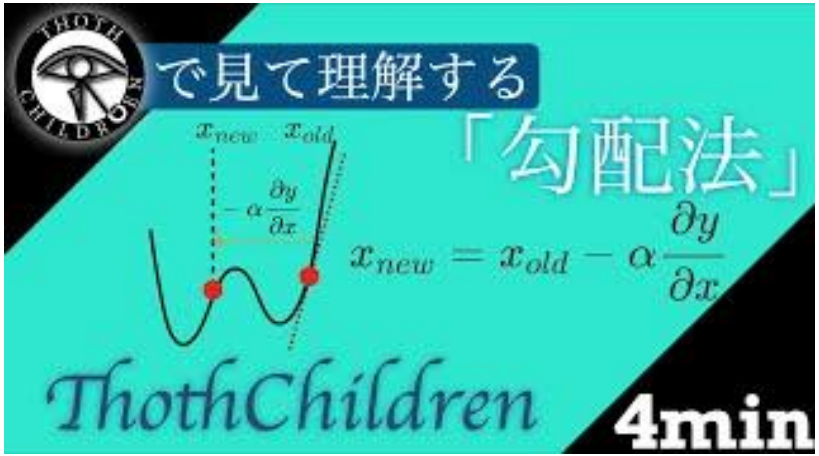
$\boldsymbol{W}_*, \boldsymbol{b}_*$ の集合 (パラメータθをもつ関数)

モデルパラメータ θ の学習



例えば $\mathcal{L}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = (\boldsymbol{y} - \hat{\boldsymbol{y}})^\top (\boldsymbol{y} - \hat{\boldsymbol{y}})$ (二乗誤差)

関数の値を最小化する θ を求める (= 学習) するヒントはすでにある！



- 前ページの例なら

$$\boldsymbol{x} \in \mathbb{R}^N \longrightarrow \boxed{\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}} \longrightarrow \boldsymbol{y} \in \mathbb{R}^M$$

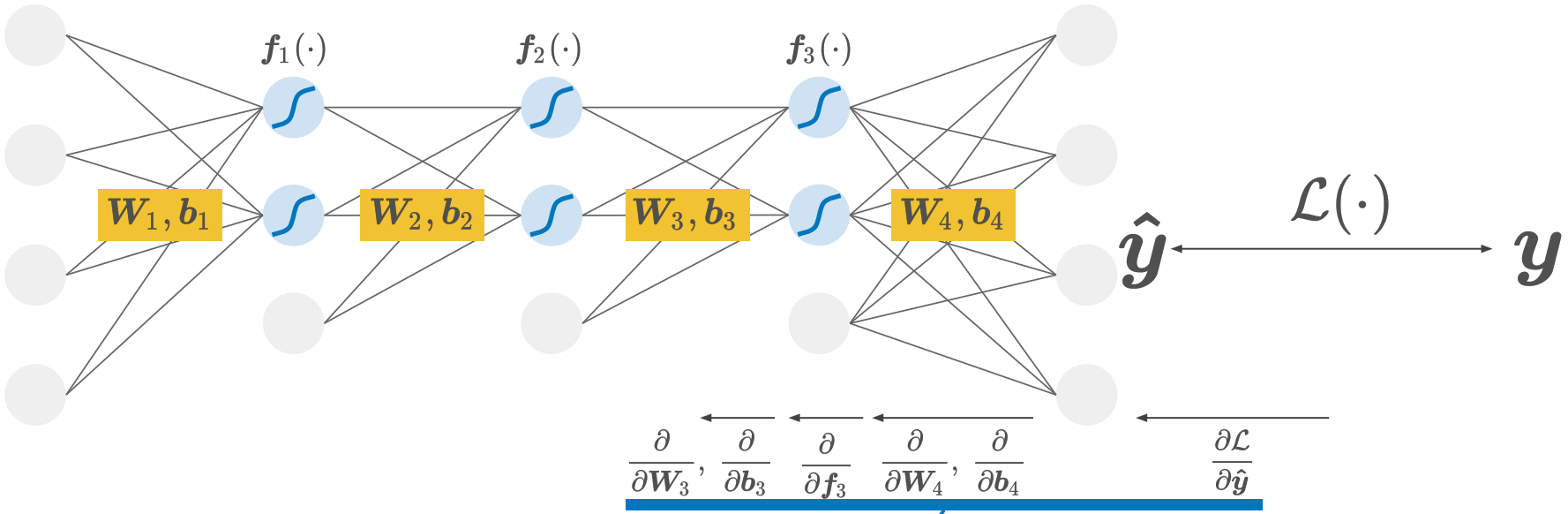
$$\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \boldsymbol{A}$$

- 連鎖律 (chain rule) も同様に適用可能
  - 信号処理システムを連結する．ニューラルネットワークを連結する

$$\boldsymbol{x} \in \mathbb{R}^N \longrightarrow \boxed{\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}} \longrightarrow \boldsymbol{y} \in \mathbb{R}^M \longrightarrow \boxed{\boldsymbol{g}(\boldsymbol{y}) = \boldsymbol{C}\boldsymbol{y} + \boldsymbol{d}} \longrightarrow \boldsymbol{z} \in \mathbb{R}^L$$

$$\frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{f}} \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}} = \boldsymbol{C}\boldsymbol{A}$$

バックプロパゲーション



連鎖律で計算可能

$$W_3 \leftarrow W_3 - \alpha \frac{\partial \mathcal{L}}{\partial W_3}$$

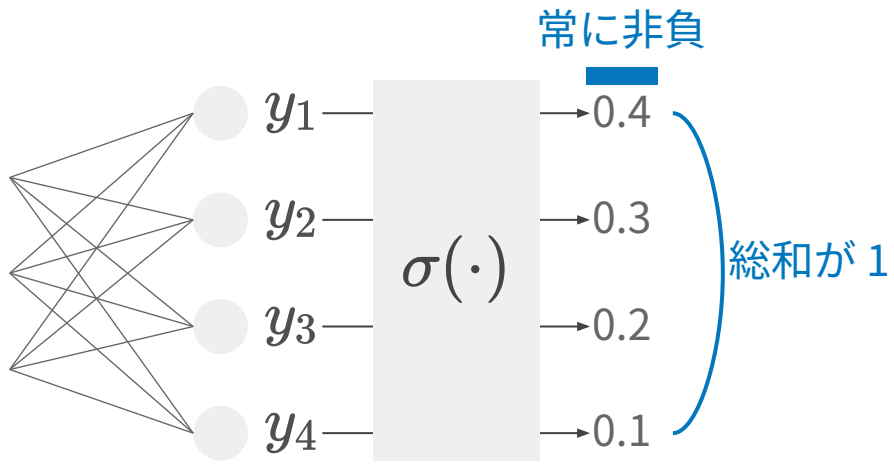
更新後　更新前　目的関数の負の勾配

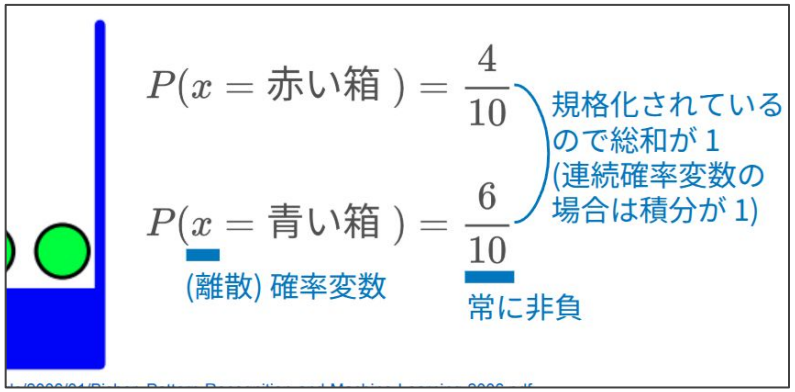
ソフトマックス関数

$$\sigma(\boldsymbol{y})_j = \frac{\exp(y_j)}{\underbrace{\sum_{\text{all } k} \exp(y_k)}}$$

exp した各要素の
割合だと解釈できる

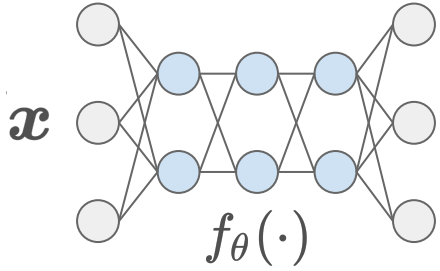
exp を作用させたベクトル全要素で正規化





条件が同じ！すなわちニューラルネットワークの出力を確率と考えることができる

ニューラルネットワークを
離散確率変数の確率分布を出力するモデルだと考える



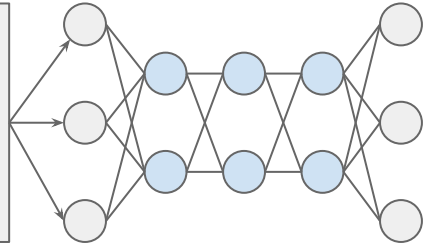
$p_\theta(\boldsymbol{y}|\boldsymbol{x})$

モデルパラメータ θ で計算される，
x に対する y の条件付き確率

“私”

単語を数値で
表現する
何らかの方法

次回以降に解説



$P(\text{は}|\text{私})$

$P(\text{を}|\text{私})$

$P(\text{に}|\text{私})$

単語列で考えてみよう

私 / は / リンゴ / を / 食べる
彼 / は / バナナ / が / 好き / だ
私 / は / 公園 / に / 行く
リンゴ / が / 落ちる
私 / は / 赤い / リンゴ / を / 買う
空 / は / 青い
私 / は / 彼 / と / リンゴ / を / 分けた
彼女 / は / メロン / を / 食べる
私 / は / リンゴ / が / 大好き / だ
犬 / が / 走って / いる

50 単語の文章 (“/” は単語境界)

“私 は” の連続 2 単語の出現確率は？

$$P(\text{私}, \text{は}) = \underbrace{P(\text{は}|\text{私})}_{\text{“私”の後に“は”が出現する確率}}\underbrace{P(\text{私})}_{\text{“私”の出現確率}}$$

$P(\text{私}) = \frac{5}{50}$ (50 単語のうち 5 回出現)

$P(\text{は}|\text{私}) = \frac{5}{5}$ (“私” に後続する 5 単語のうち 5 回出現)

連続 3 単語以上にも拡張可能

$$P(\text{私}, \text{は}, \text{彼}) = P(\text{彼}|\text{は}, \text{私})P(\text{は}|\text{私})P(\text{私})$$

条件付き確率を計算する，
パラメータ Θ を持つニューラルネット

まとめ

まとめ

- 確率
  - 条件付き確率，同時確率，連鎖律，単語列で確率を考える
- 線形代数
  - ベクトル・行列の和・積，一次変換，アフィン変換
- 微分
  - ベクトルを微分，ベクトルで微分
- ニューラルネットワーク
  - 多層パーセプトロン，確率値を出力するニューラルネットワーク

課題

- https://github.com/takamichi-lab/nlp-lecture-keio/tree/main/2026/codes/02 にアクセスせよ．

- 以降のページに示す python コードを実装し，コードと出力結果を載せた pdf を提出せよ．

- LLM や code assist を使用しても構わないが，まずは自分で解くこと．
  - 使用した場合は，どう使用してどう反映したかも pdf に書くこと．
  - 使用したにも関わらず使用に関する記載がない場合は 0 点とする．

文字 n-gram (連続する n 文字) の確率を計算せよ

data.txt

```
いあいいあういいういいいうあううあいいいいううい
あういあああああいいういいあいいううあうあいあい
いああうあいうあいあういうういいういあいういうあ
うああういううううあいあういいいいあいいういあああ
```

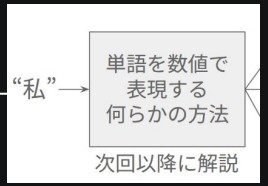
```python
# 課題： data.txt を読み込んで，文字の n-gram (n = 1, 2) を
# 表示するプログラムを作成してください．
# すでに書かれている箇所は変更しても構いません．

def n_gram(text, n):
    """テキストを n-gram に分割する関数"""


if __name__ == "__main__":
    """メイン関数"""

    unigram = n_gram(text, 1)
    bigram = n_gram(text, 2)

    # 1-gram (unigram, ユニグラム)
    print("あ", unigram["あ"])
    print("い", unigram["い"])
    print("う", unigram["う"])

    # 2-gram (bigram, バイグラム)
    print("あ あ", bigram["あ あ"])
    print("あ い", bigram["あ い"])
    print("あ う", bigram["あ う"])
    print("い あ", bigram["い あ"])
```

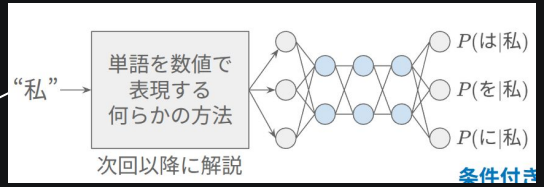
“あ” が出現する確率
(“あ”の出現回数 / 全文字数)

“あ” と “あ” が連続して出現する確率

条件付き確率を計算するニューラルネットワークを構築せよ

```python
# "あ", "い", "う" のいずれかを入力とし、
# 次元数 3 の確率値を出力するニューラルネットワークを構築してください
# ニューラルネットワークの構成，使用する Python モジュールは問いません.
# 学習は不要で，重みは適当に（例えば乱数値に）設定してください.
# 例：入力 "あ" に対して，出力 [0.7, 0.2, 0.1] を得る.
# すでに書かれている箇所は変更しても構いません.

input_vec = {
    "あ": [1, 0, 0],
    "い": [0, 1, 0],
    "う": [0, 0, 1]
}

def neural_network(input_char):
    """ニューラルネットワークの関数"""

    return output_vec # 3次元の確率値を返す


if __name__ == "__main__":
    """メイン関数"""

    output_a = neural_network("あ")
    output_i = neural_network("い")
    output_u = neural_network("う")

    print("入力: あ, 出力:", output_a) # 全要素は非負の値で，合計は 1 でなければならない
    print("入力: い, 出力:", output_i)
```





学習は不要．すなわち，
“あ” “い” “う” の入力から
確率値の条件を満たす
何らかの値が出ればOK